



OULUN YLIOPISTO
UNIVERSITY of OULU

Explainable AI for Log-Based Anomaly Detection in Security Monitoring: Reasoning Pipelines and Cross-Dataset Evaluation.

University Of Oulu
Faculty of Information Technology and
Electrical Engineering
Master's Thesis

Emmanuel Ikwunna
9th March 2026

Abstract

Abstract is needed to sum the master's thesis up. The abstract is to be uploaded into Optima before the final grading of the thesis. Please find the current information about the format given in Optima.

The guide includes instructions for students. It is written keeping in mind the idea that the user may utilise it e.g. by pasting his or her text on the current text. The contents include information about formatting the text, positioning tables and figures, among other things. In addition, the use of proper literature is instructed. Even if there is no strict structure for the thesis, a recommendation is offered in this guideline.

One important guideline for the text is that do not write too short paragraphs. For instance, if there is only one sentence in a paragraph, the sentence must be really important and influential to form a paragraph of its own.

It is not possible to provide information in a guideline like this for all issues related to master's thesis. For example, the research process, ways to acquire research material and its analysis are excluded in the guideline. On the other hand, a structure for a research plan is provided in the appendices.

Keywords

first keyword, second keyword, other keywords

Supervisor

Title, position First name Last name

Foreword

The foreword is not instructed by the supervisors. In other words, the student may write in this section what she or he wants to share with readers. However, it is a custom to thank all those who have contributed to the research somehow. When acknowledging people, their affiliations are given (e.g. Professor, University Lecturer, Adjunct Professor, Mrs.) This guideline is based on the previous version that was written in Finnish and finalised by Dr. Lasse Harjumaa in January 2007. This version is to replace the earlier version. I want to thank all those people who have contributed to the earlier versions and this newest version, the first written in English. Hopefully this guideline will serve both students and faculty with its instructions that include both formal and informal regulations and recommendations. In the first phase, the constructive comments are received with pleasure by raija.halonen@oulu.fi. Oulu, January 10, 2011

Raija Halonen Oulu, March 10, 2020

Contents

Abstract	2
Foreword.....	3
Contents.....	4
1 Introduction.....	5
2 Literature Review	6
2.1 Log-Based Anomaly Detection	6
2.1.1 Traditional Approaches	6
2.1.2 Deep Learning Era	7
2.2 Large Language Models (LLMs) in Cybersecurity	9
2.2.1 Capability Shift: From Pattern Matching to Semantic Reasoning...	9
2.2.2 Applications	9
2.2.3 Challenges	10
2.3 Explainable AI (XAI) for Time-Series & Security	11
2.3.1 Post-Hoc Methods	11
2.3.2 Attention Mechanisms	11
2.3.3 Rationale Generation.....	12
2.3.4 The Research Gap.....	12
2.4 Summary & Position of Methodology	13
3 Methodology	15
3.1 System Overview	15
3.1.1 Forward Pass	16
3.1.2 Inference	16
3.2 SecBERT Encoder	17
3.2.1 Input Processing and Pooling.....	17
3.2.2 Update Strategy	17
3.3 Alpha-Gating Mechanism	17
3.3.1 Gating Architecture	18
3.3.2 Interpretability Properties.....	18
3.4 Training Objective: Alignment and Sparsity	19
3.5 Three-Stage Training Strategy	19
3.5.1 Stage 1: Decoder Adaptation (Supervised Fine-Tuning).....	20
3.5.2 Stage 2: Projector Alignment.....	20
3.5.3 Stage 3: End-to-End Fine-Tuning	21
3.6 Datasets.....	21
3.6.1 AIT Log Data Set v1.1	21
3.6.2 OpenSSH Dataset	21
3.6.3 BGL Dataset.....	21
3.7 Evaluation Metrics	21
3.7.1 Classification Performance	21
3.7.2 Fidelity Score.....	21
3.7.3 Hallucination Rate	21
3.7.4 Alpha Sparsity.....	21
References	22
Appendix A Structure for the research plan	26

1 Introduction

In the thesis we follow the style introduced by The American Psychological Association (APA). The APA style can be found easily in the Internet and some sites provide a quick guide, too. E.g. http://www.waikato.ac.nz/library/learning/g_apaguide.shtml and <http://owl.english.purdue.edu/owl/resource/560/01/> are useful links.

It is important to follow given instructions. In academic theses, not only the content but also the format is important. Generally every academic publication forum requires that the publications follow their guidelines. In the theses accepted in the Department of Information Processing Science the format is APA. Currently there are several editions published from APA. The general rule is that the latest available edition is applied. Currently the newest edition is 6th. If a thesis is already in process it is not needed to transfer it into a newer edition of APA. Whichever you apply, do it consistently.

In addition to teach the students to follow given formal instructions, the guideline aims to unify and standardise the outlook of the theses made in the department. The guideline also enables the supervisors to focus on the content of the theses as the students already consider the outlook and format themselves. In this sense, it is a question of available resources for supervision and guidance.

The use of language and grammar cannot be discussed in detail in this kind of guide. However, the writing style should meet the general academic writing styles in the sense that no causeries are accepted or other lightweight texts such as jokes or rumblings. In other words, in academic theses all writing must be appropriate and reasonable. There are several guidebooks for academic writing available in the Oulu University Library, for example, and in the Internet. For those who write their thesis in Finnish there are books such as *Tieteellinen kirjoittaminen*. The style reference by APA (American Psychological Association, 2010) offers fruitful practical hints for writing thesis in English.

As the guideline is written according to the instructions, it enables the students to copy their text (without format) on the document and thus get their text into the right format. The format is to be used in the Bachelor's Theses and in the Master's Theses. In case of other theses, essays or reports it is recommended that the students inquire their teachers if the guideline is to be followed or not.

The structure of the guideline is as follows. The formal instructions for different topics are presented next. This is followed by examples of references and their use. After that the structure of theses and its writing style is discussed briefly. The guideline ends with a summary.

2 Literature Review

This chapter surveys the foundational concepts and existing research that underpin the proposed Sec-LogLLM architecture. It begins with an examination of log-based anomaly detection, tracing the evolution from traditional rule-based and statistical methods to modern deep learning approaches. The review then explores the integration of Large Language Models into cybersecurity, discussing their capabilities and practical deployment challenges. Finally, the chapter examines Explainable AI techniques relevant to security applications, identifies the research gap that motivates this work, and positions the proposed methodology within the existing literature.

2.1 Log-Based Anomaly Detection

Log files provide detailed, chronological records of system events, including warnings, errors, configuration changes, and user actions [1, 2, 3, 4]. As modern systems grow in scale and complexity, logs have become essential for performance monitoring, security auditing, transaction tracing, and fault diagnosis [5]. Log anomaly detection aims to identify deviations from normal log patterns that may indicate malicious activity or system malfunctions. Detecting such anomalies is critical for maintaining system reliability and mitigating the financial and operational risks associated with cyberattacks, which are estimated to cost trillions of euros annually [6].

Log analysis, and consequently anomaly detection, faces several significant challenges primarily driven by the nature and scale of the data. First, modern systems generate logs in real time and at massive scale, often exceeding 50 GB or 120–200 million lines per hour in large services, making manual inspection and traditional processing infeasible [7, 4]. Second, Logs are typically unstructured or semi-structured text files generated by logging statements in source code [2]. Because developers are allowed to write free-text messages, the format and semantics of logs vary significantly across systems, leading to high-dimensional features with complex interrelationships. This complexity and diversity increase the difficulty of accurate anomaly detection [8]. Finally, for anomaly detection to be useful, it must be timely, requiring decisions to be made in a streaming fashion to allow users to intervene in ongoing attacks or performance issues. Offline methods that require multiple passes over the entire log data are unsuitable for real-time security monitoring [2].

2.1.1 Traditional Approaches

Early log anomaly detection efforts relied heavily on human expertise [5]. As the volume of logs grew, research shifted toward automated, data-driven methods, broadly categorized into rule-based systems and statistical approaches, many of which depend on logs first being converted into a structured format through a process known as log parsing [8].

Parsing

Log parsing as discussed earlier represents the initial stage of preprocessing in the log analysis pipeline, it aims to produce structured input for ML algorithms. The primary role of log parsing is structural extraction, this process converts the raw log content into a structured event description. Drain is an effective online log parser designed to process logs accurately and efficiently in a streaming and timely manner. It incrementally cluster incoming log messages by identifying their constant (template) structure and separating variable parts (parameters). To accelerate the searching process needed to match a new log

to an existing log group, Drain utilizes a fixed depth parse tree [8]. Other parsers exist, such as the earlier mentioned Spell, which uses Longest Common Subsequence (LCS) technique to dynamically identify and update log patterns [9].

Rule-based & Statistical Methods

Rule-based methodologies were among the first attempts to automate log analysis to reduce human error. These methods typically rely on explicitly defined rules, patterns, or known indicators of abnormal behavior, often requiring specific domain knowledge from human experts [10]. However, relying solely on keywords or structural features often prevents a large portion of log anomalies from being detected and can lead to unnecessary alarms (alarm fatigue) if the system constantly evolves [10, 11].

Invariant mining is another traditional approach that captures co-occurrence patterns between different log keys [2, 11]. This method defines a window (time or session based) and detects whether certain mined quantitative relationships, or invariants, hold true within that window. Furthermore, invariant mining is typically characterized as an unsupervised offline method.

Statistical methods leverage mathematical principles to identify normal patterns and flag deviations statistically likely to be anomalous. These methods typically operate on numeric vector representations of logs, often using only log keys and their counts rather than parameter values [12, 2]. PCA is a linear transformation technique that converts correlated variables into uncorrelated principal components [7, 13]. In log analysis, PCA operates at the session level, where log entries are grouped by identifiers and converted into event count vectors recording log key frequencies [12].

Clustering algorithms group similar data instances to identify patterns or outliers [14, 13]. LogCluster is an unsupervised method that groups textually similar log messages using IDF-scaled vectorization to build a knowledge base of normal and abnormal clusters [12, 5].

Following traditional statistical and rule-based methods, log anomaly detection quickly adopted general Machine Learning (ML) algorithms. Support Vector Machines (SVM) is a flexible model capable of classification and outlier detection. One-Class SVM (OCSVM) variant is trained only on normal data to identify anomalies that fall outside a learned boundary [15, 13]. Random Forests (RF) is an ensemble method that builds multiple decision trees on random subsets of features and data [16, 17]. Isolation Forest (iForest) is a tree-based unsupervised algorithm that isolates observations distinct from the remaining data [18, 7, 13].

2.1.2 Deep Learning Era

Deep learning approaches use neural networks to automatically learn representations from log sequences, treating logs analogously to natural language. These models typically follow a pipeline of preprocessing, parsing, vectorization, and neural network classification [19].

Sequential Models

DeepLog, proposed by Du et al. (2017) [2], uses Long Short-Term Memory networks to learn normal execution patterns through a forecasting-based self-supervised approach. The model predicts the probability distribution of the next log key given a history of

preceding keys derived from parsing tools like Spell [9, 10]. An incoming log key is classified as normal if it falls within the top g candidates with the highest predicted probabilities, enabling detection at the granular per-log-entry level [5].

Transformer Models

LogBERT, proposed by Guo et al. (2021), applies Bidirectional Encoder Representations from Transformers (BERT) to log anomaly detection. It encodes each log key using bidirectional self-attention, capturing dependencies across entire sequences [20]. LogBERT is trained solely on normal log data using two self-supervised objectives: Masked Log Key Prediction (MLKP) and Volume of Hypersphere Minimization (VHM) [5, 7, 20].

LogGPT leverages the Generative Pre-trained Transformer (GPT-2) architecture, which is based on the Transformer decoder. This decoder-only structure is inherently autoregressive and operates strictly unidirectionally [21]. During inference, the model detects anomalies based on its forecasting accuracy. A sequence is flagged as abnormal if the actual incoming log key is not included in the Top-K predicted log keys generated at that position.

Attention mechanisms enable models to dynamically weigh the importance of different elements within input sequences [22]. In Transformer models like LogBERT, multi-head self-attention enables each log key to attend to every other key, producing context-aware embeddings [20]. Beyond Transformers, attention has been integrated into LSTM-based models like LogRobust for improved robustness [10, 5].

Deep learning models consistently outperform traditional statistical methods and classical ML approaches, with superior F1 scores attributed to their ability to learn semantic embeddings and model complex sequential structures [1]. Table 1 compares representative models across benchmark datasets.

Model	Architecture Type	Log Parser Dep.	HDFS F1	BGL F1	T-bird F1
LogCTBL [5]	Hybrid (CNN-TCN-Bi-LSTM + BERT)	Yes (Drain3)	-	0.9987	0.9978
LAnoBERT [7]	Transformer (BERT-MLM, Parser-free)	No	0.9645	0.8749	0.9990
OneLog [19]	End-to-End HCNN (Character-based)	No	0.9900	0.9900	0.9900
LogFiT [1]	Transformer (Fine-tuned BERT)	No	0.9497	0.9122	0.9414
DeepLog [2]	LSTM-based (Forecasting)	Yes (Spell)	0.7734	0.8612	0.9308
LogBERT [20]	Transformer (BERT-MLKP/VHM)	Yes (Drain)	0.8232	0.9083	0.9664
LogFormer [4]	Transformer (Pre-train/Tuning)	Yes (Drain)	0.9800	0.9700	0.9900
LogGPT [21]	Transformer (GPT-2 Causal)	Yes (Drain)	0.9010	0.9580	0.9860

Table 1: Comparison of representative log anomaly detection models.

Limitation: These models prioritize predictive performance (e.g., F1 scores) but often function as opaque ”black boxes,” failing to provide actionable explanations for why a

specific log sequence is flagged as anomalous. Furthermore, index-based architectures (e.g., DeepLog) discard rich textual semantics by converting logs into rigid templates. Even advanced reconstruction-based models can suffer from the "identical shortcut" problem, where they learn to reconstruct anomalies as well as normal data, leading to detection failures that aggregate metrics like F1 may mask [23].

2.2 Large Language Models (LLMs) in Cybersecurity

The integration of Large Language Models (LLMs) into cybersecurity represents a paradigm shift from rigid, rule-based systems to dynamic, semantic-aware analysis. While traditional deep learning methods often struggle to capture the rich information embedded in log data organized as natural language, LLMs leveraging transformer architectures offer superior language comprehension abilities [24].

2.2.1 Capability Shift: From Pattern Matching to Semantic Reasoning

Historically, log analysis relied on the invariant assumption, which presumes that log templates do not change and that matching keywords (e.g., "error") is sufficient for detection [10]. This approach often fails because software evolution leads to frequent changes in logging statements[25].

- **Semantic Understanding:** Unlike traditional approaches that rely on log templates, which can result in a loss of contextual information, LLMs enable a move toward semantic reasoning. For instance, models like LogFiT use BERT to learn the linguistic structure and sequential patterns of normal log data, allowing the system to "understand" the syntax and semantics of logs rather than just matching fixed patterns [26].
- **Contextual Embeddings:** Pre-trained models generate contextualized word embeddings, where a token's representation changes based on its surrounding context. This allows the model to distinguish between semantically different events even if they share similar words, for example, distinguishing between "VM Create finished" and "VM Fatal error" [10].
- **Generalization:** LLMs demonstrate symbolic reasoning capabilities that extend beyond semantic priors, allowing them to adapt to new input-label mappings and handle unstable log data where new log events emerge over time [27].

2.2.2 Applications

The application of LLMs in cybersecurity extends beyond simple anomaly detection to complex tasks requiring synthesis and generation.

- **Threat Intelligence Summarization:** LLMs can process unstructured data, such as threat reports and logs, to extract meaningful insights [28]. Tools like HuntGPT integrate LLMs to present threat insights in an easily understandable format via a conversational agent, improving analyst comprehension of alerts. This capability is critical for Security Operations Centers (SOCs), where analysts must triage massive volumes of data [29].
- **Vulnerability Detection:** LLMs are increasingly used to map vulnerabilities (CVEs) to adversarial tactics (e.g., MITRE ATT&CK). By fine-tuning models like SecBERT, systems can classify how a vulnerability might be exploited based on

its description. Furthermore, advanced models like SecureBERT 2.0 support automated vulnerability assessment by interpreting complex document structures and source code artifacts [30].

- **Automated Remediation and Patching:** Generative models can assist in remediation by producing actionable responses and code. For example, specialized models like StarCoder2 focus on generating code, which is essential for automated patching workflows. In practical deployments, LLMs can guide users through penetration testing procedures and generate necessary code steps for attacks or defenses. This capability reduces manual effort and speeds up the remediation of identified threats [29].

2.2.3 Challenges

Despite their potential, deploying LLMs in live cybersecurity environments introduces significant hurdles regarding performance and reliability.

2.2.3.1 1. Computational Cost and Inference Latency

- **Latency:** Inference latency is a critical bottleneck for real-time log streams. While traditional log analysis tools struggle with volume, LLM-based solutions face high latency due to the complexity of transformer architectures. For instance, using the OpenAI API can result in latencies ranging from hundreds of milliseconds to seconds, which is often too slow for real-time AIOps scenarios [25].
- **Resource Intensity:** Privacy-preserving inference (PPI) for transformers is particularly slow; for example, a BERT-base model can take over 70 seconds to inference a single sample using Secure Multi-Party Computing (SMPC), compared to less than a second for plaintext inference. This computational heaviness makes deployment on edge devices or in latency-sensitive environments difficult [31].

2.2.3.2 2. Token Limits and Context Windows

- **Context Restrictions:** Transformer models have a limit on the number of tokens they can process at once, known as the context window (e.g., 512 tokens for standard BERT, 4096 for Longformer) [26].
- **Analysis Fragmentation:** Long log sequences often exceed these limits. A single log sequence can contain numerous templates, generating an excessive number of tokens that surpass the model's memory capacity. This restricts the analysis of long-term dependencies in system behavior unless specialized architectures like Longformer are employed to handle larger log paragraphs [1].

2.2.3.3 3. Hallucination and the Need for Grounding

- **Inventing Attacks:** A major risk is hallucination, where LLMs confidently generate incorrect or non-existent information. In a security context, a model might invent attacks that did not happen or provide incorrect remediation advice [31].
- **Fine-tuning Risks:** Interestingly, fine-tuning LLMs on new knowledge can sometimes increase hallucinations because the model may struggle to acquire new factual knowledge that conflicts with its pre-training [32].
- **Grounding (RAG):** To mitigate this, there is a need for grounding via Retrieval-Augmented Generation (RAG). RAG systems retrieve relevant,

up-to-date information (such as specific log entries or threat intelligence) to provide the LLM with factual context, thereby reducing hallucinations and improving the reliability of the output [14].

2.3 Explainable AI (XAI) for Time-Series & Security

Explainable Artificial Intelligence (XAI) encompasses methods that help users interpret and understand how machine learning models make their decisions, thereby improving transparency [27, 19]. The need for explainability is paramount in high-stake security applications such as anomaly detection [33, 19]. A fundamental challenge in implementing AI, particularly in high-stakes domains, is managing the tension between model accuracy and interpretability [34].

2.3.1 Post-Hoc Methods

LIME (Local Interpretable Model-agnostic Explanations) provides explanations for individual model predictions by learning a simple, interpretable surrogate model locally around the specific prediction instance [35]. The core process involves perturbing the input instance to generate new data samples, obtaining predictions from the black-box model, and training a simple model on the weighted samples. While LIME is efficient, the authors note that its explanations can sometimes be unreliable.

SHapley Additive exPlanations (SHAP) is a post-hoc method that generates local explanations for the predictions of any ML model [33] by viewing the explanation process through the lens of game theory [34]. It assigns an importance value (the Shapley value) to each feature, quantifying its contribution to the model's output.

Feature	LIME	SHAP
Methodology	Perturbation-based local approximation.	Game-theoretic (Shapley values); additive attribution.
Scope	Local explanation for a single instance.	Local explanation, but derived from a framework designed for global coherence.
Model Dependency	Model-agnostic.	Model-agnostic (using approximations).
Theoretical Basis	Minimizes local fidelity and complexity loss.	Unique solution satisfying local accuracy, missingness, and consistency.
Computational Cost	Generally faster for local explanations.	Computationally slower (though approximations exist).

Table 2: Comparison of LIME and SHAP methodologies.

2.3.2 Attention Mechanisms

Transformer models rely on attention mechanisms that allow models to selectively focus on different parts of the input sequence to compute representations [33, 27]. These attention weights can be leveraged as a form of explanation by quantifying the importance assigned by the model [27]. Several tools have been developed to map attention scores back onto input data, such as BERTViz [36]. The use of attention weights as explanations is controversial, with researchers debating their faithfulness and stability ("Attention is not Explanation") [37].

2.3.3 Rationale Generation

Rationale generation involves prompting Large Language Models (LLMs) to articulate the reasoning behind their predictions, a technique prominently known as Chain-of-Thought (CoT) prompting [27, 14]. Unlike standard prompting which seeks a direct answer, CoT encourages the model to decompose complex problems into a series of intermediate reasoning steps, effectively mimicking the slower, logical "System 2" mode of human thinking [14]. This capability can be elicited via few-shot prompting, where the model is provided with examples of reasoning, or zero-shot prompting using phrases such as "Let's think step-by-step" [14].

In the cybersecurity domain, researchers have adapted this approach to creating a "Log Chain-of-Thought" (LCoT) for anomaly detection [4]. Rather than asking the model to detect an anomaly directly, LCoT guides the model to first extract log templates and then determine if anomalies exist based on the relationship between templates and variables; this multi-hop approach has been shown to improve the F1 score of models like ChatGPT compared to direct decision generation [4]. To further enhance reliability, techniques such as Self-Consistency involve sampling multiple reasoning paths and selecting the final answer via majority vote, while Tree-of-Thoughts (ToT) allows the model to explore and evaluate multiple branching reasoning paths [14].

However, the validity of these rationales is contested. Critics argue that generated explanations may be plausible post-hoc rationalizations, functioning as invented stories that justify an action, rather than faithful representations of the actual neural processes that produced the prediction [37]. Experiments with counterfactual prompts suggest that intermediate reasoning steps often guide the model to replicate patterns rather than adhere to strict factual logic, raising concerns about the faithfulness of CoT in high-stakes environments [27].

2.3.4 The Research Gap

There is a lack of models that provide *intrinsic, real-time* explanations for log anomalies without requiring expensive post-hoc analysis. Current literature largely bifurcates into two extremes: highly accurate but opaque deep learning models (e.g., LogBERT, DeepLog) and interpretable but less capable shallow models [20, 2]. Security practitioners often resort to "post-hoc" explainability techniques like LIME or SHAP [33]. However, these methods suffer from critical flaws in a security context:

1. **Latency:** Generating perturbations for SHAP requires thousands of inference passes, making it prohibitive for real-time SOC monitoring.
2. **Fidelity:** Post-hoc explanations are approximations. They do not reveal the exact decision path the model took, leading to a "trust gap" for analysts.

Developers must therefore choose between designing intrinsically interpretable models (ante-hoc explanation) or applying post-hoc methods to black-box models. This work targets the missing middle: a model that is *intrinsically* explainable by design, where the "explanation" (the alpha weights) is computed deeply within the forward pass itself [33, 23].

2.4 Summary & Position of Methodology

The preceding review reveals three key findings that collectively define the limitations of log anomaly detection methodologies discussed in this chapter. First, deep learning models such as LogBERT [20], DeepLog [2], and LogFormer [4] have demonstrated strong predictive performance, consistently achieving F1 scores above 0.90 on benchmark datasets (Table 1). However, these models function as opaque “black boxes,” offering no insight into *why* a particular log sequence was flagged as anomalous [23]. Second, post-hoc explainability techniques such as LIME [35] and SHAP [34] can be applied to any model but introduce significant computational overhead. SHAP, for instance, requires thousands of perturbation-based inference passes per prediction, rendering it impractical for real-time Security Operations Centre (SOC) monitoring [33]. Furthermore, these methods produce only approximate explanations that do not faithfully represent the model’s internal decision process. Third, while Large Language Models have demonstrated promising semantic reasoning capabilities for security analysis, they face practical deployment challenges including inference latency [25], context window limitations [1], and a tendency to hallucinate non-existent threats [32].

To address these compounding limitations, this thesis proposes **Sec-LogLLM**, a hybrid architecture designed to resolve the tension between detection performance and interpretability. Rather than treating anomaly detection and explainability as separate concerns, Sec-LogLLM integrates three complementary components into a unified pipeline:

1. **Domain-Specific Semantic Encoding:** A cybersecurity-specialised encoder (SecBERT) extracts contextualised representations from raw log text, capturing domain-specific semantics that template-based approaches like Drain discard [8]. This addresses the semantic gap identified in index-based architectures such as DeepLog, which reduce logs to rigid key sequences [23].
2. **Alpha-Gating Mechanism:** A novel trainable projector assigns a scalar importance weight ($\alpha \in (0, 1)$) to each log line within a sequence. These weights provide an *intrinsic*, mathematically grounded importance map that identifies exactly which events contributed to the model’s decision. Crucially, unlike SHAP or LIME, these explanations are computed within the forward pass itself at zero additional inference cost [33].
3. **LLM-Based Reasoning:** A causal language model decoder receives the α -weighted log representations and generates structured, human-readable analysis including attack classification, MITRE ATT&CK mapping, and natural language reasoning. This directly addresses the “trust gap” that analysts experience with opaque detection systems [29].

Table 3 summarises how Sec-LogLLM is positioned relative to the approaches reviewed in this chapter.

Approach	Accuracy	Explainability	Real-Time	Parser-Free
DeepLog / LogBERT	High	None	✓	×
LIME / SHAP (Post-hoc)	N/A	Approximate	×	N/A
LLM Direct Prompting	Moderate	Rationale	×	✓
Sec-LogLLM	High	Intrinsic (α)	✓	✓

Table 3: Positioning of Sec-LogLLM relative to existing approaches.

By optimising these components end-to-end through a three-stage training strategy, Sec-LogLLM targets the “missing middle” identified in the research gap: a model that is intrinsically explainable by design, where the explanation is not an afterthought but an integral part of the prediction pipeline. The following chapter presents the detailed architecture, mathematical formulations, and training methodology of the proposed system.

3 Methodology

This chapter presents the architecture, mathematical formulations, and training methodology of Sec-LogLLM, the proposed hybrid system for explainable log-based anomaly detection. The design rationale for each component is grounded in the research gaps identified in Chapter 2.

3.1 System Overview

Sec-LogLLM is a hybrid architecture that combines a domain-specific encoder with a causal language model decoder through a novel explainability bottleneck. The system is designed to accept raw, unstructured log text as input and produce a structured JSON analysis containing three fields: a binary classification label (`Attack` or `Normal`), an optional MITRE ATT&CK technique code (`mitre_t_code`), and a natural language reasoning string explaining the decision. Crucially, the system simultaneously produces per-log importance weights (α values) that identify which log lines in the input sequence drove the classification.

The architecture consists of three components arranged as a sequential pipeline:

1. **SecBERT Encoder:** A cybersecurity-specialised BERT model (`jackaduma/SecBERT`) that processes each raw log line independently and produces a fixed-size embedding vector $\mathbf{h}_i \in \mathbb{R}^{768}$. Variable-length log sequences containing up to 128 lines are encoded into a padded tensor of shape $(B, T, 768)$, where B is the batch size and T is the maximum sequence length in the batch.
2. **Alpha-Gated Projector:** A trainable module that serves two functions simultaneously. First, it linearly projects each 768-dimensional encoder embedding into the decoder’s embedding space ($\mathbb{R}^{d_{\text{dec}}}$), bridging the dimensionality gap between the two models. Second, it computes a scalar importance weight $\alpha_i \in (0, 1)$ for each log line via a learned gating network, and multiplies the projected embedding by this weight. The output dimension of the projector is automatically set to match the decoder’s hidden size during initialisation.
3. **LLM Decoder:** A causal language model (`Qwen/Qwen2.5-3B-Instruct`) that receives a composite input embedding sequence and autoregressively generates the structured JSON output. The decoder is adapted to the security domain via Low-Rank Adaptation (LoRA) rather than full fine-tuning, preserving the model’s general language capabilities while specialising its output behaviour.

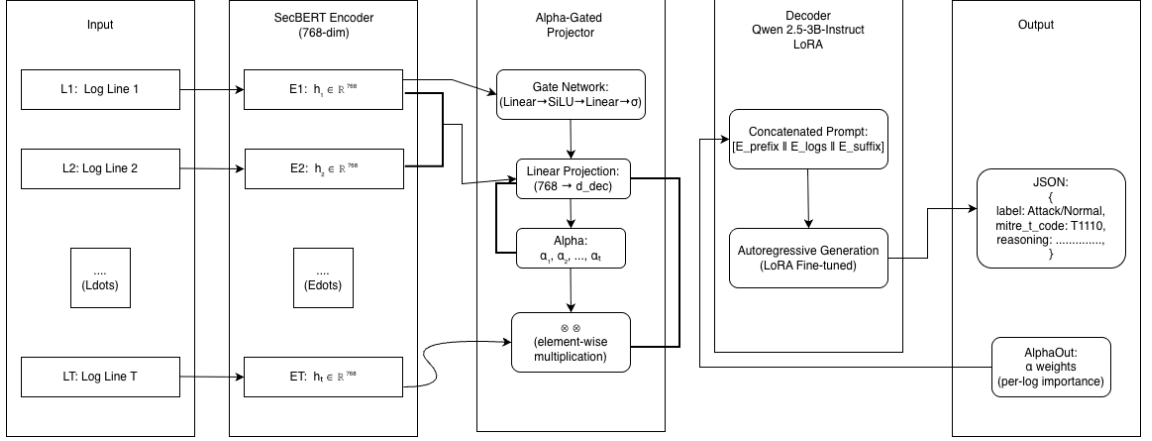


Figure 1: Architecture of Sec-LogLLM. Raw log lines are encoded by SecBERT, weighted by the alpha-gating mechanism, and decoded by the LLM into a structured JSON analysis.

3.1.1 Forward Pass

During the forward pass, the three components are composed into a single end-to-end pipeline. Given a batch of raw log sequences, the system first encodes each log line through SecBERT to obtain pooled embeddings. These embeddings are then passed through the alpha-gated projector, which produces both the projected representations and the corresponding alpha weights.

The projected log embeddings are not passed to the decoder in isolation. Instead, they are sandwiched between two segments of tokenised text that form a structured prompt. The *prefix* segment contains a system instruction that establishes the model’s role as a Security Operations Centre (SOC) copilot, followed by a user-turn preamble introducing the log embeddings. The *suffix* segment closes the user turn, specifies the desired JSON output format, and opens the assistant turn to begin generation. These text segments are converted into decoder-space embeddings using the decoder’s own embedding layer.

The final input to the decoder is a concatenation of four embedding sequences along the sequence dimension:

$$\mathbf{E}_{\text{input}} = [\mathbf{E}_{\text{prefix}} \parallel \mathbf{E}_{\text{logs}} \parallel \mathbf{E}_{\text{suffix}} \parallel \mathbf{E}_{\text{target}}] \quad (1)$$

where $\mathbf{E}_{\text{prefix}}$ and $\mathbf{E}_{\text{suffix}}$ are the tokenised prompt embeddings, $\mathbf{E}_{\text{logs}}$ are the alpha-weighted projected log embeddings, and $\mathbf{E}_{\text{target}}$ contains the ground-truth response embeddings during training (omitted during inference). A corresponding attention mask is constructed to account for padding in variable-length sequences. During training, labels are set to -100 (ignored by the cross-entropy loss) for all positions except the target response tokens, ensuring the decoder only learns to generate the analysis output.

3.1.2 Inference

At inference time, the system operates identically through the encoding and projection stages, but the target embeddings are excluded from the concatenated input. The decoder then autoregressively generates up to 128 new tokens, which are decoded back to text using the decoder’s tokeniser. The generated text is expected to be a valid JSON object conforming to the output schema. Alongside the generated text, the system returns the alpha weights $\alpha \in \mathbb{R}^{B \times T \times 1}$ for each log line, enabling analysts to inspect which events were deemed most relevant to the decision.

3.2 SecBERT Encoder

To extract rich semantic meaning from unstructured security logs before passing them to the LLM, the architecture employs a specialised encoding stage. Rather than using standard pre-trained models like BERT or RoBERTa, which are optimised for general English NLP tasks, Sec-LogLLM integrates `jackaduma/SecBERT`, a domain-specific BERT variant pre-trained extensively on cybersecurity text and system logs. Using a domain-specific encoder prevents the semantic dilution that occurs when generic tokenisers indiscriminately split complex identifiers (e.g., file paths, IP addresses, or registry keys) into meaninglessly small subwords.

3.2.1 Input Processing and Pooling

Raw log sequences are passed to the encoder in variable-length batches. Each individual log line l_i within a sequence is independently tokenised. To balance computational efficiency with contextual capacity, the maximum sequence length per log line is capped at 256 tokens; any line exceeding this limit is truncated, while shorter lines are padded up to the batch maximum.

Once tokenised, the log lines pass through the bidirectional Transformer layers of SecBERT. To reduce the sequence of token embeddings into a single fixed-size representation for the entire log line, the model extracts the embedding corresponding to the special `[CLS]` token from the final hidden state. Formally, for a given log line l_i , the encoding process yields a pooled embedding vector:

$$\mathbf{h}_i = \text{SecBERT}(l_i)_{[\text{CLS}]} \quad \text{where} \quad \mathbf{h}_i \in \mathbb{R}^{768} \quad (2)$$

This fixed-size representation $\mathbf{h}_i$ encapsulates the semantic state and syntax of the single event, making it suitable for subsequent mapping into the LLM’s larger embedding space.

3.2.2 Update Strategy

Unlike some parameter-efficient fine-tuning approaches that freeze early layers of an encoder to retain lower-level syntactic knowledge, Sec-LogLLM employs an all-or-nothing updating strategy across the entire encoder block depending on the training stage. During the initial alignment phase (Sec. 3.5), the entire SecBERT module is unfrozen, allowing the gradients to flow through all of its layers. This ensures that the encoder can learn to align its intermediate representations with the LLM’s latent space. Once this alignment is complete, the entire encoder is explicitly frozen (requiring no gradients) during the final end-to-end generation phase, ensuring that the carefully aligned representations do not drift while the projector and LLM adapters are optimised for the final task.

3.3 Alpha-Gating Mechanism

In standard transformer-based systems, passing encoder embeddings directly to a decoder via a simple linear projection achieves feature alignment but sacrifices interpretability. In the context of log-based anomaly detection, it is not sufficient for a model to simply flag a sequence of 128 logs as malicious; analysts require immediate evidence regarding exactly which log lines triggered the alert. While post-hoc explainability methods such as SHAP or LIME can identify these influential features, they require thousands of expensive perturbation-based inference passes per input. This latency is prohibitive for real-time Security Operations Centre (SOC) monitoring, and

such methods often suffer from fidelity issues, producing approximations rather than faithful reflections of the model’s true reasoning process.

To address this, Sec-LogLLM uses an *Alpha-Gating Mechanism*—a learned computational bottleneck placed between the encoder and decoder. This mechanism intrinsically learns the importance of each log line during training and computes it directly during the forward pass, yielding faithful feature attribution with zero computational overhead at inference time.

3.3.1 Gating Architecture

The alpha-gating mechanism is implemented as a parallel branch augmenting the standard cross-modal linear projection. Given a 768-dimensional encoder embedding $\mathbf{h}_i \in \mathbb{R}^{d_{\text{enc}}}$ for the i -th log line, a standard approach simply maps it to the decoder’s hidden dimension d_{dec} via a linear projection:

$$\tilde{\mathbf{z}}_i = \mathbf{W}_{\text{proj}} \mathbf{h}_i + \mathbf{b}_{\text{proj}} \quad (3)$$

Optional regularisation steps such as LayerNorm and Dropout may be applied after the projection, though in the default configuration both are disabled.

In the alpha-gated architecture, a secondary gating network independently computes a scalar weight $\alpha_i \in (0, 1)$ representing the relevance of the specific log line. To capture complex relationships within the embedding space, the gating network is designed as a two-layer multi-layer perceptron (MLP) employing a SiLU (Sigmoid Linear Unit) activation function in the hidden layer.

Formally, the intermediate hidden representation $\mathbf{g}_i \in \mathbb{R}^{d_{\text{gate}}}$ (where $d_{\text{gate}} = 128$) and the scalar logit $s_i \in \mathbb{R}$ are computed as:

$$\mathbf{g}_i = \text{SiLU}(\mathbf{W}_1 \mathbf{h}_i + \mathbf{b}_1) \quad (4)$$

$$s_i = \mathbf{W}_2 \mathbf{g}_i + \mathbf{b}_2 \quad (5)$$

Here, $\mathbf{W}_1 \in \mathbb{R}^{128 \times d_{\text{enc}}}$, $\mathbf{W}_2 \in \mathbb{R}^{1 \times 128}$, and $\mathbf{b}_1, \mathbf{b}_2$ are learned bias terms.

To derive the final gating value α_i , the raw logit s_i is divided by a temperature scaling parameter τ (default $\tau = 1.0$) before passing through a sigmoid activation function:

$$\alpha_i = \sigma\left(\frac{s_i}{\tau}\right) = \frac{1}{1 + \exp(-s_i/\tau)} \quad (6)$$

This gating value α_i is then element-wise multiplied with the projected embedding $\tilde{\mathbf{z}}_i$ to produce the final, alpha-weighted representation $\mathbf{z}_i$ that is passed to the LLM decoder:

$$\mathbf{z}_i = \alpha_i \cdot \tilde{\mathbf{z}}_i \quad (7)$$

3.3.2 Interpretability Properties

The resulting α_i weights offer an *intrinsic* explanation of the pipeline’s behaviour by acting as a mathematically grounded attention map. Because $\mathbf{z}_i \approx \mathbf{0}$ when $\alpha_i \rightarrow 0$, log lines that the gate network determines to be task-irrelevant are effectively zeroed out, masking their semantic content from the decoder’s self-attention layers. Conversely, log lines with $\alpha_i \rightarrow 1$ pass their full projected semantic information to the language model.

Crucially, because the α_i values are computed natively inside the projector module, these importance scores are effortlessly extracted alongside the generated JSON response in a

single forward pass. This bypasses the severe computational bottlenecks inherent to post-hoc strategies like SHAP, offering exact, unapproximated insight into the precise feature subset the unified pipeline relied upon to detect an anomaly.

3.4 Training Objective: Alignment and Sparsity

The alpha-gating mechanism described in Section 3.3 requires a specialised loss function during the projector alignment phase (Stage 2). A naive approach would simply minimise the reconstruction error between the projected log embeddings and some target representation. However, without an explicit incentive to keep α values sparse, the gating network would learn to assign uniformly high importance to all log lines, rendering the explainability mechanism uninformative.

To enforce a “need-to-know” principle—where only genuinely relevant log lines receive high α values—the training objective combines two complementary terms. The first is a *Mean Squared Error* (MSE) alignment loss that minimises the distance between the mean-pooled projected embeddings and the mean-pooled decoder-space target embeddings:

$$\mathcal{L}_{\text{align}} = \text{MSE} \left(\frac{1}{T} \sum_{i=1}^T \mathbf{z}_i, \frac{1}{S} \sum_{j=1}^S \mathbf{e}_j^{\text{target}} \right) \quad (8)$$

where $\mathbf{z}_i$ are the alpha-weighted projected log embeddings and $\mathbf{e}_j^{\text{target}}$ are the decoder-space embeddings of the ground-truth target response, with the summations producing sequence-level representations.

The second term is an L_1 sparsity penalty on the mean alpha values, which penalises the model for assigning unnecessary importance to log lines:

$$\mathcal{L}_{\text{sparsity}} = \frac{1}{N} \sum_{i=1}^N |\alpha_i| \quad (9)$$

where N is the total number of log lines in the batch. The complete training objective for Stage 2 is therefore:

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{align}} + \lambda_{L_1} \cdot \mathcal{L}_{\text{sparsity}} \quad (10)$$

The sparsity coefficient λ_{L_1} controls the trade-off between alignment fidelity and interpretability. A default value of $\lambda_{L_1} = 0.01$ is used, which was determined through preliminary experiments to encourage meaningful sparsity without degrading alignment quality.

3.5 Three-Stage Training Strategy

Training the full Sec-LogLLM pipeline from scratch in a single pass would be unstable: the randomly initialised projector would inject noise into the decoder, while the decoder’s gradients would disturb the encoder’s pre-trained representations. To address this, the system is trained in three sequential stages, each with a distinct objective and a carefully chosen set of frozen and trainable parameters. Table 4 summarises the configuration of each stage.

Stage	Encoder	Projector	Decoder	Loss	LR
1: Decoder SFT	Frozen	Frozen	LoRA trained	Cross-entropy	2×10^{-5}
2: Alignment	Trained	Trained	Frozen	$\mathcal{L}_{\text{total}}$ (Eq. 10)	2×10^{-4}
3: End-to-End	Frozen	Trained	LoRA trained	Cross-entropy	1×10^{-4}

Table 4: Summary of the three-stage training strategy. “Frozen” indicates all parameters have `requires_grad=False`. “LoRA trained” indicates that only the low-rank adapter parameters are updated.

3.5.1 Stage 1: Decoder Adaptation (Supervised Fine-Tuning)

The first stage adapts the pre-trained LLM decoder to the security domain and the desired output format before it interacts with the encoder or projector. At this stage, neither the SecBERT encoder nor the alpha-gated projector participates in training; both are frozen. The objective is to teach the decoder to produce well-structured JSON responses containing an attack classification label, a MITRE ATT&CK technique code (where applicable), and a natural language reasoning string.

Rather than full fine-tuning, which would require updating all 3 billion parameters of the Qwen2.5-3B-Instruct model, Stage 1 employs Low-Rank Adaptation (LoRA). LoRA injects small, trainable low-rank matrices into specific layers of the transformer, dramatically reducing the number of trainable parameters while preserving the model’s pre-trained knowledge. The LoRA configuration used is as follows:

- **Rank** (r): 16
- **Alpha** (α_{LoRA}): 32
- **Dropout**: 0.1
- **Target modules**: `q_proj`, `k_proj`, `v_proj`, `o_proj`, `gate_proj`, `up_proj`, `down_proj`

The training data consists of individual log entries formatted as ChatML conversations, where the system message establishes the model’s role as a security analyst, the user message presents a raw log line, and the assistant message contains the desired JSON response. The model is trained using the standard causal language modelling cross-entropy loss with a learning rate of 2×10^{-5} over 3 epochs, with gradient accumulation across 4 steps per update.

3.5.2 Stage 2: Projector Alignment

With the decoder now capable of producing well-formatted security analyses, Stage 2 establishes the communication channel between the SecBERT encoder and the decoder through the alpha-gated projector. The Stage 1 LoRA adapter is loaded into the decoder, and the entire decoder (including the adapter weights) is frozen. The encoder and projector are both unfrozen and trained jointly.

The objective is to align the projected encoder representations with the decoder’s latent space using the composite loss function defined in Section 3.4. To compute the alignment loss, the sequence of projected log embeddings $\mathbf{z}_1, \dots, \mathbf{z}_T$ is mean-pooled along the sequence dimension to produce a single vector, and similarly, the decoder-space target embeddings are mean-pooled. The MSE between these two pooled

representations, combined with the L_1 sparsity penalty on the alpha values, drives the encoder and projector to learn representations that are both compatible with the decoder’s expectations and interpretably sparse.

This stage trains for 2 epochs with a learning rate of 2×10^{-4} , a batch size of 8, and gradient accumulation over 4 steps. The AdamW optimiser is used with $\beta = (0.9, 0.95)$ and weight decay of 0.01. Upon completion, both the encoder and projector state dictionaries are saved as a checkpoint.

3.5.3 Stage 3: End-to-End Fine-Tuning

The final stage jointly optimises the projector and the decoder’s LoRA adapters to maximise task performance. The Stage 1 LoRA adapter and Stage 2 encoder/projector checkpoint are both loaded. The encoder is then re-frozen to prevent the aligned representations from drifting, while the projector remains trainable to allow further refinement.

To selectively unfreeze only the LoRA parameters within the decoder (rather than all decoder weights), the system scans all named parameters and sets `requires_grad=True` exclusively for those whose names contain the substring “lora”. All other decoder parameters remain frozen.

The loss function in this stage is the standard causal language modelling cross-entropy loss, as computed by the decoder model itself. This directly optimises the quality of the generated JSON output, including classification accuracy, MITRE ATT&CK code assignment, and reasoning quality. Training runs for 1 epoch with a learning rate of 1×10^{-4} , a batch size of 4, and gradient accumulation over 8 steps. The final checkpoint saves the encoder, projector, and LoRA adapter states.

3.6 Datasets

3.6.1 AIT Log Data Set v1.1

3.6.2 OpenSSH Dataset

3.6.3 BGL Dataset

3.7 Evaluation Metrics

3.7.1 Classification Performance

3.7.2 Fidelity Score

3.7.3 Hallucination Rate

3.7.4 Alpha Sparsity

References

- [1] Crispin Almodovar, Fariza Sabrina, Sarvnaz Karimi and Salahuddin Azad. ‘Can Language Models Help in System Security? Investigating Log Anomaly Detection using BERT’. In: *Proceedings of the 20th Annual Workshop of the Australasian Language Technology Association*. Ed. by Pradeesh Parameswaran, Jennifer Biggs and David Powers. Adelaide, Australia: Australasian Language Technology Association, Dec. 2022, pp. 139–147. URL: <https://aclanthology.org/2022.alta-1.19/>.
- [2] Min Du, Feifei Li, Guineng Zheng and Vivek Srikumar. ‘DeepLog: Anomaly Detection and Diagnosis from System Logs through Deep Learning’. In: *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security*. CCS ’17. Dallas, Texas, USA: Association for Computing Machinery, 2017, pp. 1285–1298. ISBN: 9781450349468. DOI: 10.1145/3133956.3134015. URL: <https://doi.org/10.1145/3133956.3134015>.
- [3] Amazon Web Services. *What are Log Files? - Log Files Explained - AWS*. <https://aws.amazon.com/what-is/log-files/>. Accessed: 2025-11-11. 2025.
- [4] Hongcheng Guo, Jian Yang, Jiaheng Liu, Jiaqi Bai, Boyang Wang, Zhoujun Li, Tieqiao Zheng, Bo Zhang, Junran peng and Qi Tian. *LogFormer: A Pre-train and Tuning Pipeline for Log Anomaly Detection*. 2024. arXiv: 2401.04749 [cs.LG]. URL: <https://arxiv.org/abs/2401.04749>.
- [5] Hong Huang, Wengang Luo, Yunfei Wang, Yinghang Zhou and Weitao Huang. ‘LogCTBL: a hybrid deep learning model for log-based anomaly detection’. In: *The Journal of Supercomputing* 81 (Jan. 2025). DOI: 10.1007/s11227-025-06926-3.
- [6] European Parliament. *Cybercrime in the EU: Threats, Trends and Policy Responses*. Tech. rep. According to an EU briefing, the annual global cost of cybercrime was estimated at approximately €5.5 trillion in recent years. Accessed November 9, 2025. European Parliamentary Research Service, 2024. URL: [https://www.europarl.europa.eu/RegData/etudes/BRIE/2024/760356/EPRS_BRI\(2024\)760356_EN.pdf](https://www.europarl.europa.eu/RegData/etudes/BRIE/2024/760356/EPRS_BRI(2024)760356_EN.pdf).
- [7] Yookyung Lee, Jina Kim and Pilsung Kang. ‘LAnoBERT: System log anomaly detection based on BERT masked language model’. In: *Applied Soft Computing* 146 (2023), p. 110689. ISSN: 1568-4946. DOI: <https://doi.org/10.1016/j.asoc.2023.110689>. URL: <https://www.sciencedirect.com/science/article/pii/S156849462300707X>.
- [8] Pinjia He, Jieming Zhu, Zibin Zheng and Michael R. Lyu. ‘Drain: An Online Log Parsing Approach with Fixed Depth Tree’. In: *2017 IEEE International Conference on Web Services (ICWS)*. 2017, pp. 33–40. DOI: 10.1109/ICWS.2017.13.
- [9] Min Du and Feifei Li. ‘Spell: Streaming Parsing of System Event Logs’. In: *2016 IEEE 16th International Conference on Data Mining (ICDM)*. 2016, pp. 859–864. DOI: 10.1109/ICDM.2016.0103.
- [10] Harold Ott, Jasmin Bogatinovski, Alexander Acker, Sasho Nedelkoski and Odej Kao. *Robust and Transferable Anomaly Detection in Log Data using Pre-Trained Language Models*. 2021. arXiv: 2102.11570 [cs.AI]. URL: <https://arxiv.org/abs/2102.11570>.

- [11] Weibin Meng, Ying Liu, Yichen Zhu, Shenglin Zhang, Dan Pei, Yuqing Liu, Yihao Chen, Ruizhi Zhang, Shimin Tao, Pei Sun and Rong Zhou. ‘LogAnomaly: Unsupervised Detection of Sequential and Quantitative Anomalies in Unstructured Logs’. In: *Proceedings of the Twenty-Eighth International Joint Conference on Artificial Intelligence, IJCAI-19*. International Joint Conferences on Artificial Intelligence Organization, July 2019, pp. 4739–4745. DOI: 10.24963/ijcai.2019/658. URL: <https://doi.org/10.24963/ijcai.2019/658>.
- [12] Shilin He, Jieming Zhu, Pinjia He and Michael R. Lyu. ‘Experience Report: System Log Analysis for Anomaly Detection’. In: *2016 IEEE 27th International Symposium on Software Reliability Engineering (ISSRE)*. 2016, pp. 207–218. DOI: 10.1109/ISSRE.2016.21.
- [13] Aurélien Géron. *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow*. 2nd. Sebastopol, CA: O’Reilly Media, 2019. ISBN: 978-1-492-03264-9. URL: <https://www.oreilly.com/library/view/hands-on-machine-learning/9781492032632/>.
- [14] Jay Alammar and Maarten Grootendorst. *Hands-On Large Language Models: Language Understanding and Generation*. Sebastopol, CA: O’Reilly Media, 2024. ISBN: 978-1-492-08828-3. URL: <https://www.oreilly.com/library/view/hands-on-large-language/9781492088283/>.
- [15] Mennatallah Amer, Markus Goldstein and Slim Abdennadher. ‘Enhancing one-class Support Vector Machines for unsupervised anomaly detection’. In: Aug. 2013, pp. 8–15. DOI: 10.1145/2500853.2500857.
- [16] Andreas C. Müller and Sarah Guido. *Introduction to Machine Learning with Python: A Guide for Data Scientists*. Sebastopol, CA: O’Reilly Media, 2016. ISBN: 978-1449369415.
- [17] Tarek Ali. ‘Next-Generation Intrusion Detection Systems with LLMs: Real-Time Anomaly Detection, Explainable AI, and Adaptive Data Generation’. Master’s thesis. Oulu, Finland: Faculty of Information Technology and Electrical Engineering, University of Oulu, 2025.
- [18] Dong Xu, Yanjun Wang, Yulong Meng and Ziyang Zhang. ‘An Improved Data Anomaly Detection Method Based on Isolation Forest’. In: Dec. 2017, pp. 287–291. DOI: 10.1109/ISCID.2017.202.
- [19] Sayedshayan Hashemi Hosseinabad. ‘Data-Driven Software System Log Anomaly Detection’. Acta Univ. Oul. A 808. Doctoral dissertation. Oulu, Finland: University of Oulu, Faculty of Information Technology and Electrical Engineering, 2025. ISBN: 978-952-62-4502-7.
- [20] Haixuan Guo, Shuhan Yuan and Xintao Wu. *LogBERT: Log Anomaly Detection via BERT*. 2021. arXiv: 2103.04475 [cs.CR]. URL: <https://arxiv.org/abs/2103.04475>.
- [21] Xiao Han, Shuhan Yuan and Mohamed Trabelsi. *LogGPT: Log Anomaly Detection via GPT*. 2023. arXiv: 2309.14482 [cs.LG]. URL: <https://arxiv.org/abs/2309.14482>.
- [22] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser and Illia Polosukhin. *Attention Is All You Need*. 2023. arXiv: 1706.03762 [cs.CL]. URL: <https://arxiv.org/abs/1706.03762>.

- [23] Runqiang Zang, Hongcheng Guo, Jian Yang, Jiaheng Liu, Zhoujun Li, Tieqiao Zheng, Xu Shi, Liangfan Zheng and Bo Zhang. *MLAD: A Unified Model for Multi-system Log Anomaly Detection*. 2024. arXiv: 2401.07655 [cs.SE]. URL: <https://arxiv.org/abs/2401.07655>.
- [24] Wei Guan, Jian Cao, Shiyu Qian, Jianqi Gao and Chun Ouyang. *LogLLM: Log-based Anomaly Detection Using Large Language Models*. 2025. arXiv: 2411.08561 [cs.SE]. URL: <https://arxiv.org/abs/2411.08561>.
- [25] Van-Hoang Le and Hongyu Zhang. ‘PreLog: A Pre-trained Model for Log Analytics’. In: *Proc. ACM Manag. Data* 2.3 (May 2024). DOI: 10.1145/3654966. URL: <https://doi.org/10.1145/3654966>.
- [26] Crispin Almodovar, Fariza Sabrina, Sarvnaz Karimi and Salahuddin Azad. ‘LogFiT: Log Anomaly Detection Using Fine-Tuned Language Models’. In: *IEEE Transactions on Network and Service Management* 21.2 (2024), pp. 1715–1723. DOI: 10.1109/TNSM.2024.3358730.
- [27] Melkamu Mersha, Khang Lam, Joseph Wood, Ali K. AlShami and Jugal Kalita. ‘Explainable artificial intelligence: A survey of needs, techniques, applications, and future direction’. In: *Neurocomputing* 599 (Sept. 2024), p. 128111. ISSN: 0925-2312. DOI: 10.1016/j.neucom.2024.128111. URL: <http://dx.doi.org/10.1016/j.neucom.2024.128111>.
- [28] Blessing Austin-Gabriel, Nurudeen Yemi Hussain, Adebimpe Bolatito Ige, Peter Adeyemo Adepoju and Adeoye Idowu Afolabi. ‘Natural language processing frameworks for real-time decision-making in cybersecurity and business analytics’. In: *International Journal of Science and Technology Research Archive* 04.02 (2023), pp. 086–095. DOI: 10.53771/ijstra.2023.4.2.0018. URL: <https://doi.org/10.53771/ijstra.2023.4.2.0018>.
- [29] Prasasthy Balasubramanian, Dumindu Kankanamge, Ekaterina Gilman and Mourad Oussalah. *AnomalyExplainer Explainable AI for LLM-based anomaly detection using BERTViz and Captum*. 2025. arXiv: 2509.00069 [cs.LG]. URL: <https://arxiv.org/abs/2509.00069>.
- [30] Yifan Liu, Shancang Li and Sarah Bin Hulayyil. ‘Intelligent Detection of Cyber Attack Patterns in Industrial IoT Using Pretrained Language Models’. In: *Electronics* 14.20 (2025). ISSN: 2079-9292. DOI: 10.3390/electronics14204094. URL: <https://www.mdpi.com/2079-9292/14/20/4094>.
- [31] Yang Li, Xinyu Zhou, Yitong Wang, Liangxin Qian and Jun Zhao. *Private Transformer Inference in MLaaS: A Survey*. 2025. arXiv: 2505.10315 [cs.CR]. URL: <https://arxiv.org/abs/2505.10315>.
- [32] Dante Shintaro Cometti and Diogo Lopes Dionisio. ‘Real World Implementation of LLM-based Log Anomaly Detection: Exploring the Feasibility of Training-Free Approaches’. Master’s thesis. University Name, 2024.
- [33] Dipkamal Bhusal, Rosalyn Shin, Ajay Ashok Shewale, Monish Kumar Manikya Veerabhadran, Michael Clifford, Sara Rampazzi and Nidhi Rastogi. ‘SoK: Modeling Explainability in Security Analytics for Interpretability, Trustworthiness, and Usability’. In: *Proceedings of the 18th International Conference on Availability, Reliability and Security*. ARES 2023. ACM, Aug. 2023, pp. 1–12. DOI: 10.1145/3600160.3600193. URL: <http://dx.doi.org/10.1145/3600160.3600193>.

- [34] Scott Lundberg and Su-In Lee. *A Unified Approach to Interpreting Model Predictions*. 2017. arXiv: 1705 . 07874 [cs.AI]. URL: <https://arxiv.org/abs/1705.07874>.
- [35] Marco Tulio Ribeiro, Sameer Singh and Carlos Guestrin. "Why Should I Trust You?": *Explaining the Predictions of Any Classifier*. 2016. arXiv: 1602 . 04938 [cs.LG]. URL: <https://arxiv.org/abs/1602.04938>.
- [36] Jesse Vig. 'BertViz: A Tool for Visualizing Multi-Head Self-Attention in the BERT Model'. In: (May 2019).
- [37] Sarah Wiegreffe and Yuval Pinter. *Attention is not not Explanation*. 2019. arXiv: 1908.04626 [cs.CL]. URL: <https://arxiv.org/abs/1908.04626>.
- [38] Risto Vaarandi. 'Mining event logs with SLCT and LogHound'. In: May 2008, pp. 1071–1074. DOI: 10.1109/NOMS.2008.4575281.
- [39] Chiung Ko, Jintaek Kang, Chaejun Lim, Donggeun Kim and Minwoo Lee. 'Application of Machine Learning Models in the Estimation of Quercus mongolica Stem Profiles'. In: *Forests* 16.7 (2025). ISSN: 1999-4907. DOI: 10.3390/f16071138. URL: <https://www.mdpi.com/1999-4907/16/7/1138>.
- [40] Dongqi Han, Zhiliang Wang, Wenqi Chen, Ying Zhong, Su Wang, Han Zhang, Jiahai Yang, Xingang Shi and Xia Yin. 'DeepAID: Interpreting and Improving Deep Learning-based Anomaly Detection in Security Applications'. In: *Proceedings of the 2021 ACM SIGSAC Conference on Computer and Communications Security*. CCS '21. ACM, Nov. 2021, pp. 3197–3217. DOI: 10 . 1145 / 3460120 . 3484589. URL: <http://dx.doi.org/10.1145/3460120.3484589>.
- [41] Suchin Gururangan, Ana Marasović, Swabha Swayamdipta, Kyle Lo, Iz Beltagy, Doug Downey and Noah A. Smith. *Don't Stop Pretraining: Adapt Language Models to Domains and Tasks*. 2020. arXiv: 2004 . 10964 [cs.CL]. URL: <https://arxiv.org/abs/2004.10964>.
- [42] Blessing Austin-Gabriel, Nurudeen Yemi Hussain, Adebimpe Bolatito Ige, Peter Adeyemo Adepoju and Adeoye Idowu Afolabi. 'Natural language processing frameworks for real-time decision-making in cybersecurity and business analytics'. In: *International Journal of Science and Technology Research Archive* 4.2 (2023), pp. 086–095. DOI: 10.53771/ijstra.2023.4.2.0018.
- [43] Christophe Bertero, Matthieu Roy, Carla Sauvanaud and Gilles Tredan. 'Experience Report: Log Mining Using Natural Language Processing and Application to Anomaly Detection'. In: *2017 IEEE 28th International Symposium on Software Reliability Engineering (ISSRE)*. 2017, pp. 351–360. DOI: 10.1109/ISSRE.2017.43.
- [44] Shaohan Huang, Yi Liu, Carol Fung, He Wang, Hailong Yang and Zhongzhi Luan. 'Improving Log-Based Anomaly Detection by Pre-Training Hierarchical Transformers'. In: *IEEE Transactions on Computers* 72.9 (2023), pp. 2656–2667. DOI: 10.1109/TC.2023.3257518.
- [45] Eduardo C. Garrido-Merchan, Roberto Gozalo-Brizuela and Santiago Gonzalez-Carvajal. 'Comparing BERT Against Traditional Machine Learning Models in Text Classification'. In: *Journal of Computational and Cognitive Engineering* 2.4 (Apr. 2023), pp. 352–356. ISSN: 2810-9570. DOI: 10 . 47852 / bonviewjcce3202838. URL: <http://dx.doi.org/10.47852/bonviewJCCE3202838>.

A Structure for the research plan

A research plan can be reported according to the next structure. The order of the items is important.

Introduction

The topic is introduced on general level. The context of the research is described and the research problem is explained and justified. The problem is situated in its larger environment. Note references when needed. The researcher may reason the topic also by describing his or her personal motivation.

Research problem and research methods

The problem under study is explained as explicitly as possible. The research problem can be divided into sub problems or presented as hypotheses. The research methods and analysis are described.

Limitations

The planned limitations and known shortcomings are reported. The reasons for them – if known – are explained from the viewpoint of the current research.

Preliminary earlier research

The prior literature is presented briefly with full sentences. All required references are included. Its relevance in the current research is described and limitations recognised in prior research are identified if possible. List of main prior literature in relation to the background theory Main background references are listed in the required format (APA).

Lähteet

Timetable

A plan to describe the planned research related to calendar time. It is recommended that the plan is discussed with supervisor to ensure enough milestones for checking thoroughly the status of the thesis.

Preliminary structure of contents

1. Introduction
2. Glossary
3. Prior research
 - (a) First
 - (b) Second
 - Subsecond
4. Sources